
Case Study

Digitalisation of Core Functions in Rogers Communications by Manisha Kriplani

Task 1: Core Functions for Digitalisation

1. Data Classification & Metadata Management

Current Challenge:

- Manual identification of sensitive PII data- Currently we are fully digital but I will describe how it started initially. So initially all the identification of customers sensitive PII- Personal Identifiable Information data was manual. All the data sampling, tagging sensitive fields, inspecting database schema by data engineers and data analysts was done manually.
- Inconsistent tagging across systems- this manual tagging after again manually identifying the sensitive data was very tedious and time consuming.

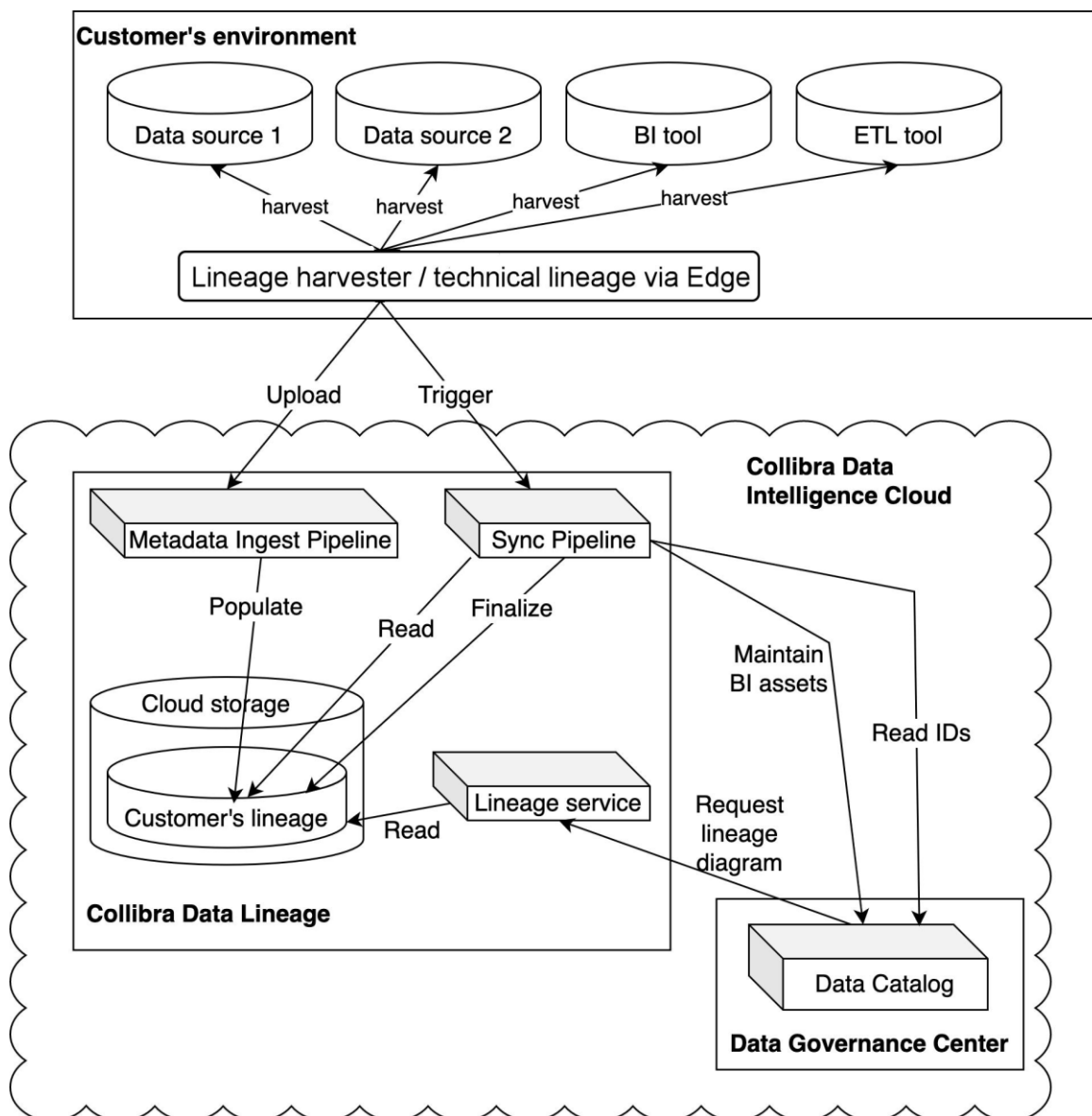
Digitalisation Approach:

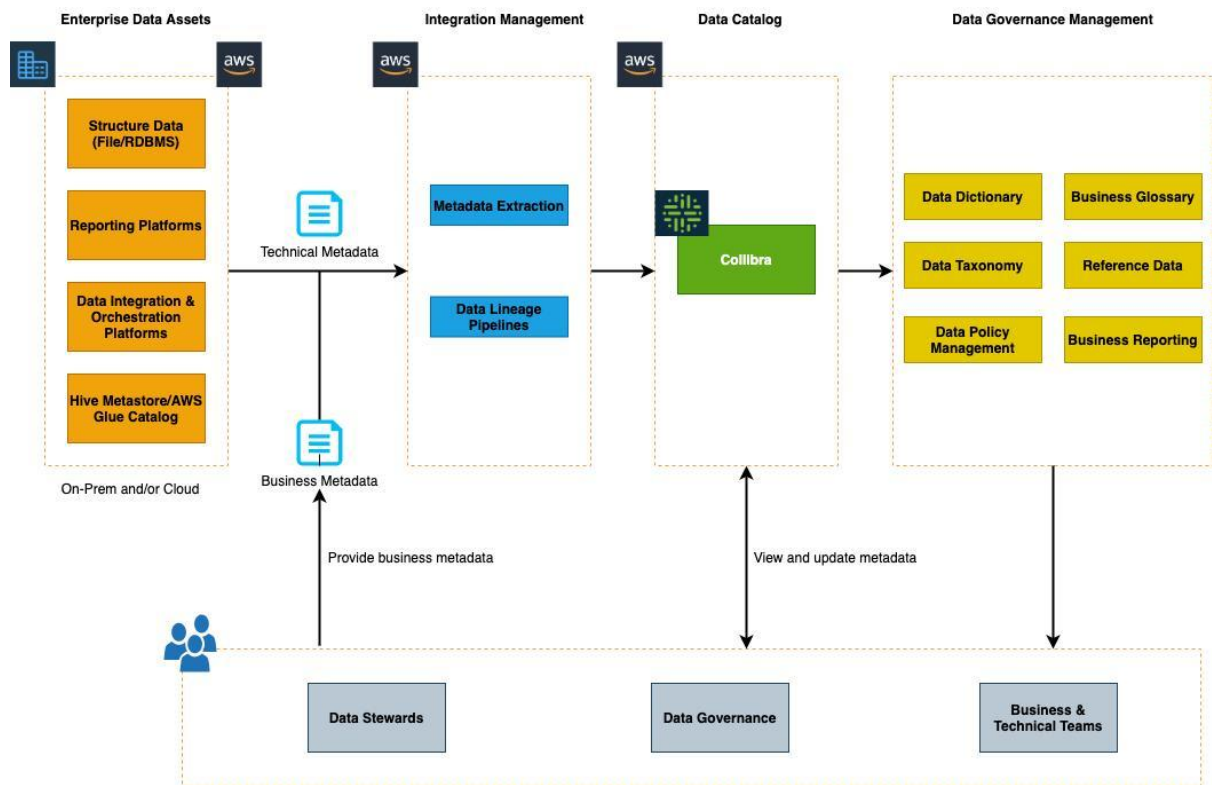
- Use of Collibra as the **central metadata repository**- we then as part of our Digitization program shifted and introduced Collibra as the central metadata repository. Collibra is a very powerful data Governance tool which helps with keeping the metadata safe at one place and segregating classified and non- classified data.
- Extract classification data via APIs- this data classification was further extracted via custome APIs which were used to connect Collibra to Snowflake and latter feed to Snowflake.
- Load into Snowflake in tables- JSON_TABLE, LANDING, LANDING_HISTORY- this extracted data was then loaded to these Snowflake tables for further processing.
- Automate tagging using **PIFAMILYGROUP & classification keys**- data engineers build the logic to automate tagging via Stored Procedures to assign proper PIFAMILYGROUP and PIFAMILYGROUPKEY to the sensitive data.

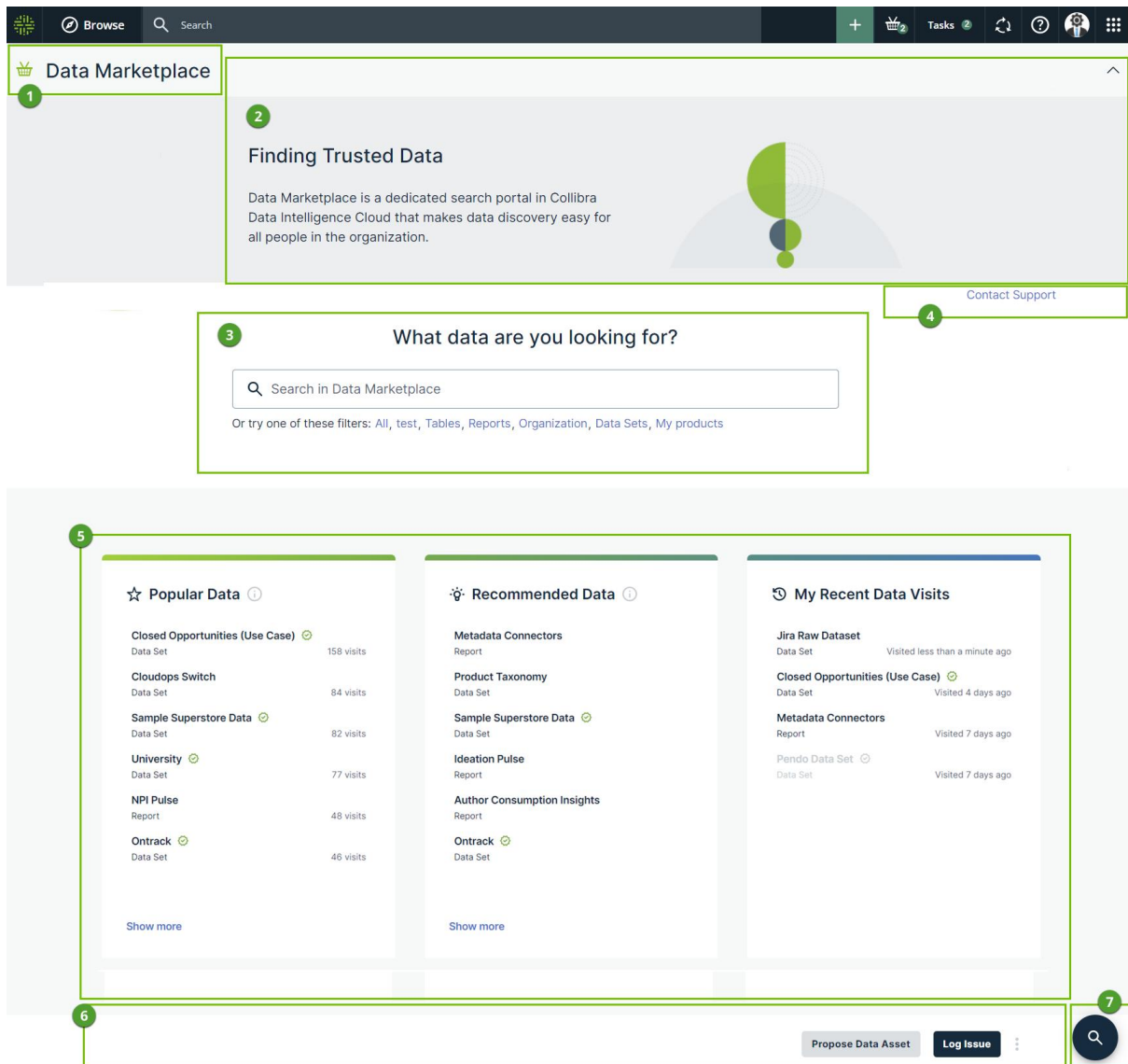
Outcome:

- Centralised and consistent classification- this resulted in tasks and procedures running in background to create a central source of metadata which acted as single source of truth. And based on it tags follow across making tagging consistent and error free.
- Real-time alignment with Collibra- we scheduled the Snowflake tasks every 30 mins for capturing real time data changes and enhancements. These tasks would call API stored procedures in background to fetch real time data changes and load it into Snowflake for further data processing and would then apply tags to the data.

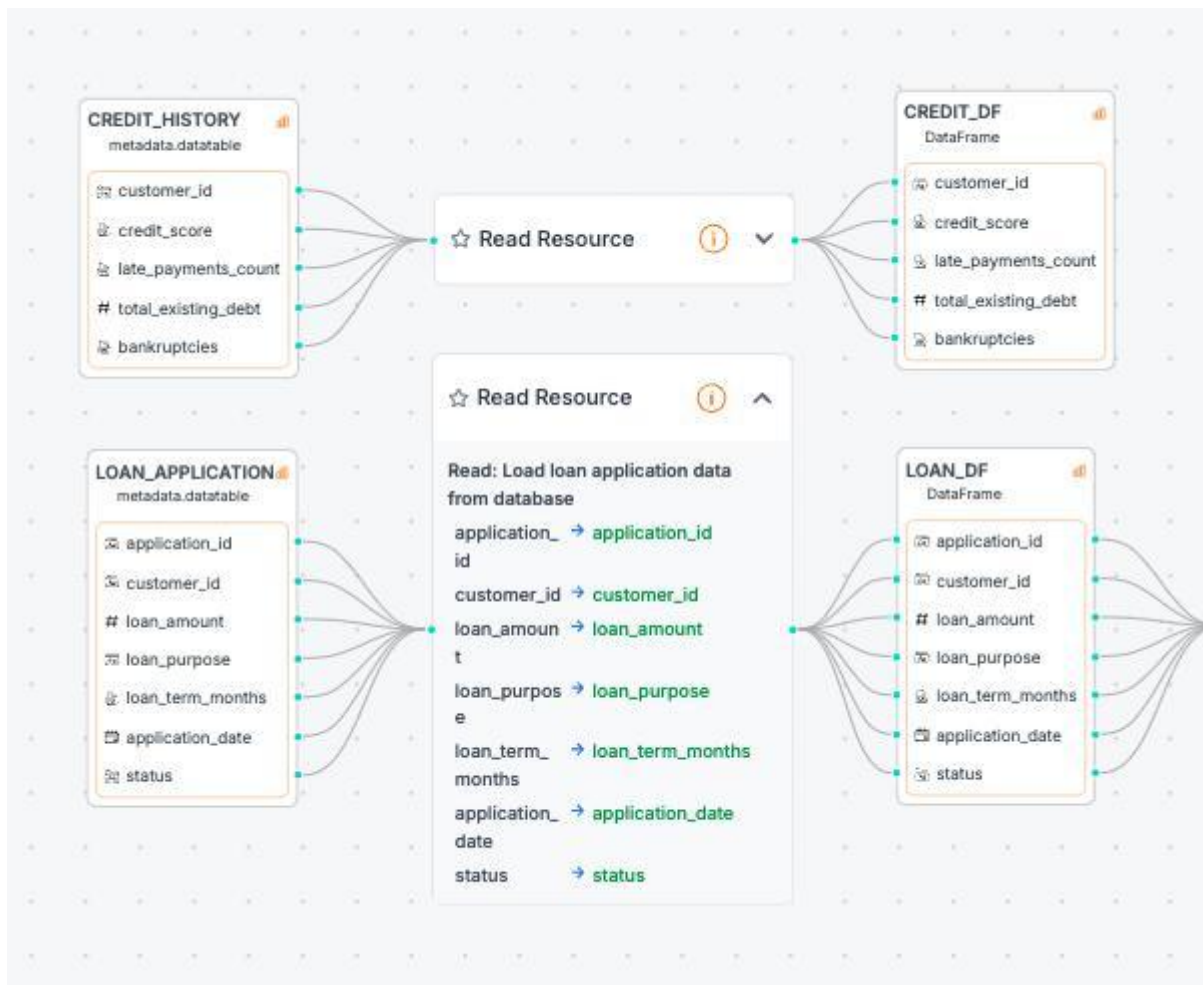
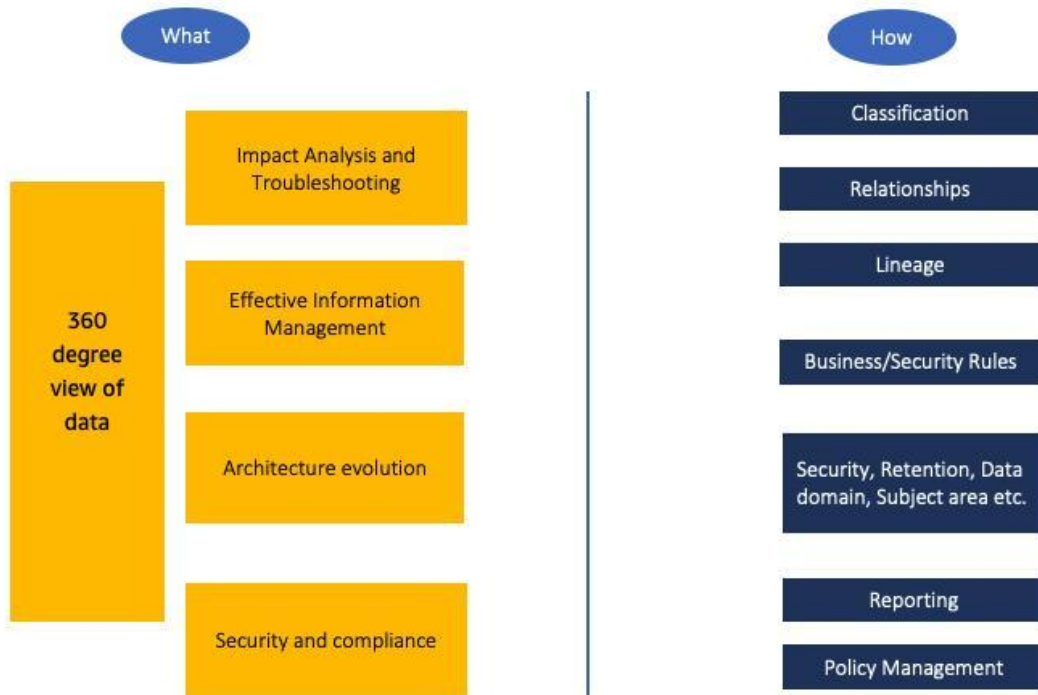
- Reduced manual effort- this fully digitized solution when went over the production environment reduced manual monitoring effort by 90% and reduced operational costs by 40%. We initially did for 20 databases at the start and eventually rolled out the release for remaining 1000+ databases spread across Rogers Canada business.
- Below diagrams illustrate how the system looked after digitization of data classification and metadata management. Everything streamlined, structured and automated through pipelines.

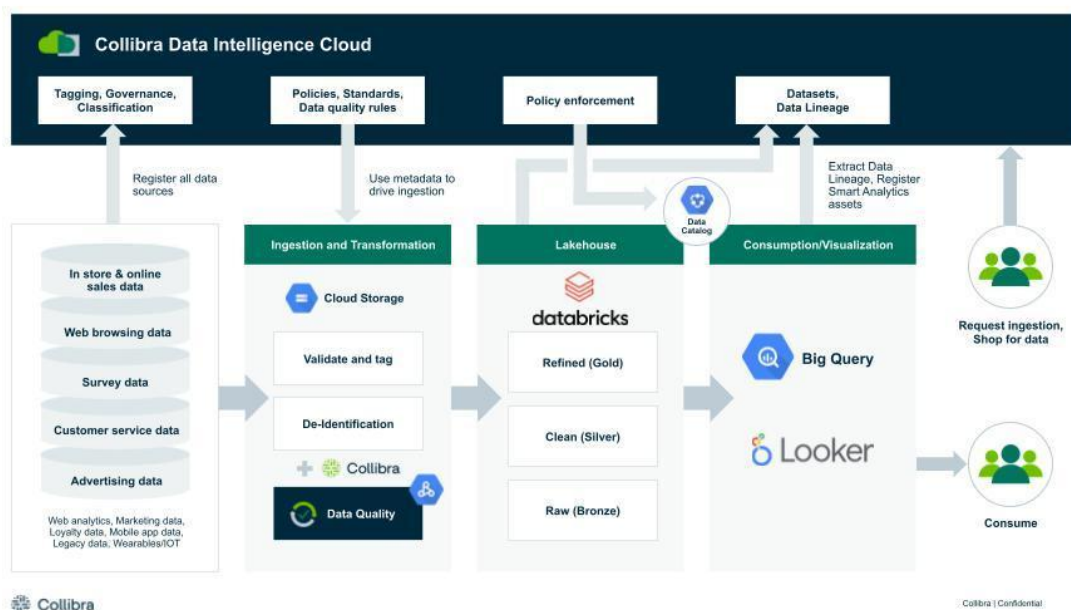
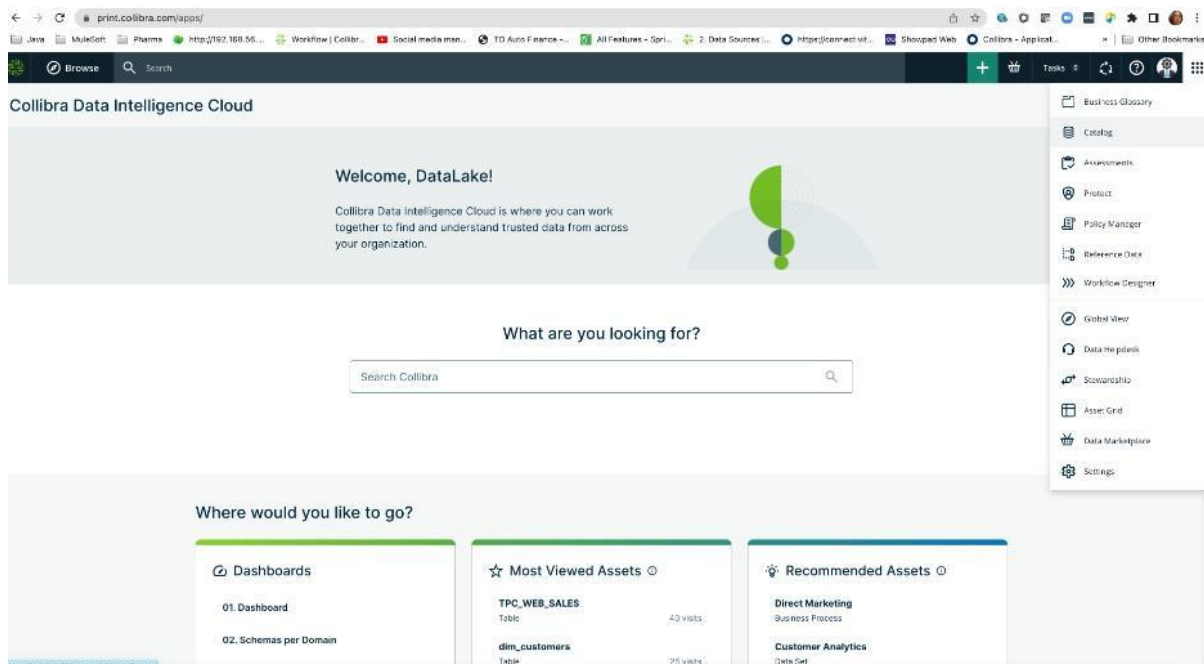






Data Governance & Metadata Management Platform





2. Data Security & Masking Enforcement

Current Challenge:

- Manual enforcement of data privacy rules- So the data engineers were manually consulting business definitions before applying those rules to the sensitive data. This created a lot of expectations from the engineers to never go wrong.
- Risk of exposing PII data- any miss in tagging the sensitive data resulted in huge business risk in managing customers sensitive data. The question came directly to Governance. How are we governing our customers data? How well are we at doing it? Is there a better way to do it? All these questions were answered when we went Digital-

Digitalisation Approach:

- Apply **dynamic masking policies in Snowflake**- how we made masking policies dynamic? We created masking policies in Snowflake using create masking policy definitions. The masking policy body was imbued with various case statements to point to different accounts and users. This accounts and users information was always kept in another table and we pointed the policy body to that table, any change in user setting resulted in automated changing the policy when the base table got altered.
- Use tagging framework:
 - AZCA - Central tags for AZCA were done centrally in the database called CONFIGURATION_AZCA_DB. What do I mean when I say central tags? Central tags means there is a single database and all other sensitive data which is located as db.schema.column_name would always follow CONFIGURATION_AZCA_DB.schema.column_name , without relocating the underlying data. These central tags ensures **consistency across enterprise**, enables **central governance control**, simplifies **policy enforcement and masking**.
 - AWS - Local tags in CONFIGURATION_DB- Rogers is divided in east and west regions in Canada from a business perspective. When we run Snowflake on top of Azure calling it AZCA region and when we run Snowflake on top of AWS we call it Canada central. So tags in AWS are local where each database can define its own tags, which are then applied to objects within that environment, enabling decentralized governance.
- Role-based and actor-based access control:
 - PII_ACCESS_DETAILS- for AZCA we have role based access controls wherein a particular user role has access to particular PII. For example PII for account_number is K1_Account_number where K1 is the key and Account_number is its value. All these key value pairs are stored in table - PII_ACCESS_DETAILS for AZCA users.
 - PII_ACCESS_DETAILS_AWS- similarly for AWS we maintain a separate table in Snowflake - PII_ACCESS_DETAILS_AWS where all the PII family key value pairs are stored for easy application of tags and dynamic masking policies.

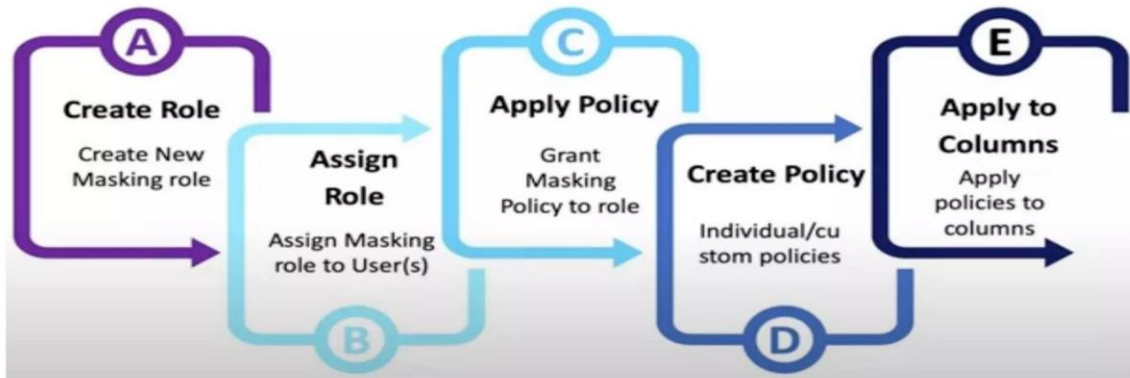
Outcome:

- Automated enforcement of security policies- All this automated workflow is set to run at scheduled intervals leading to automated enforcement of all the PII security policies at run time.
- Compliance with data privacy regulations- We very well follow GDPR and HIPAA regulations and others while being compliant and maintain the privacy of our data.
- Secure access based on roles- As we follow role based and actor based access controls the sensitive classified data is always secured and privacy level maintained.
- Again below figures illustrate how the system looked when we shifted to automating the governance platform for data security and masking enforcement-

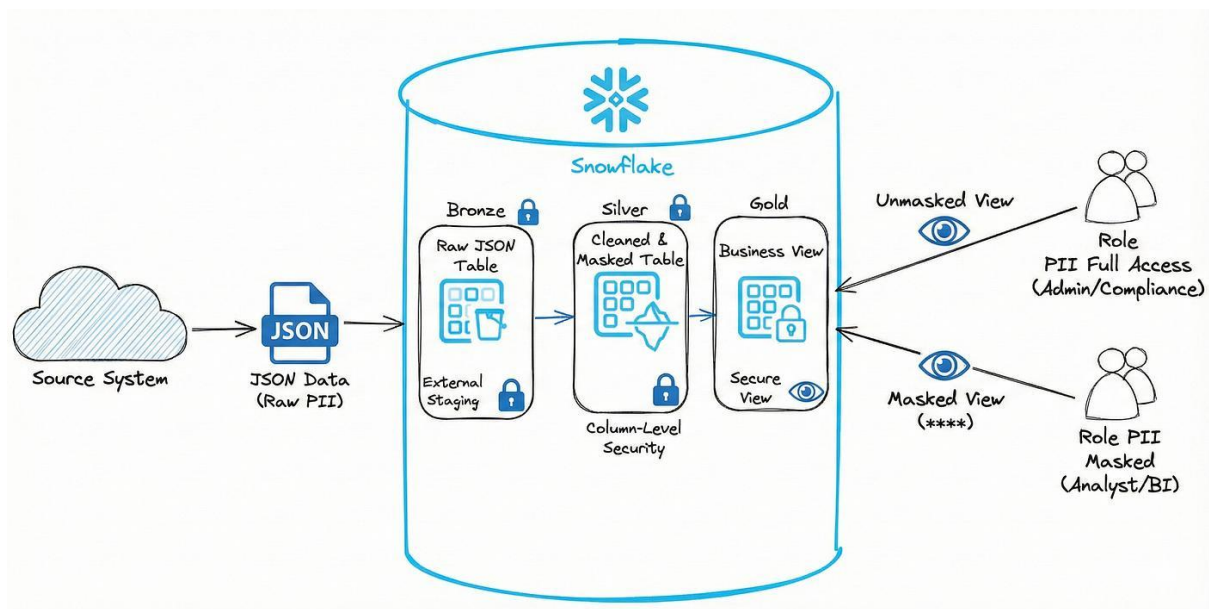
Snowflake - Dynamic Data Masking Setup

Dynamic Data Masking helps to mask table's column plain-text data masked data. This is schema level objects and currently can be applied on table & view only. Dynamic Data Masking feature is logical and applied at query time. It doesn't mask data persistently in database. Depending on the masking policy conditions, the SQL execution context, and role hierarchy, Snowflake query operators may see the plain-text value, a partially masked value, or a fully masked value.

The following figure shows the process of setting up Dynamic Data Masking in Snowflake objects.



2





COLUMN-LEVEL SECURITY

ID	Email	Salary
1	****	
2	name@example	70000

Masking Policies

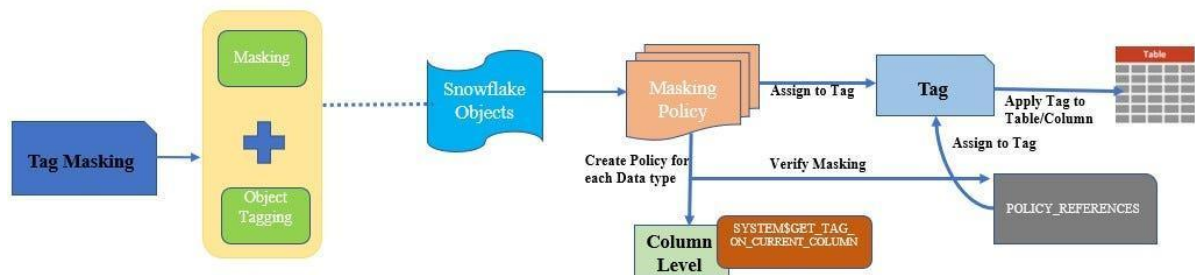
Restrict access to sensitive columns

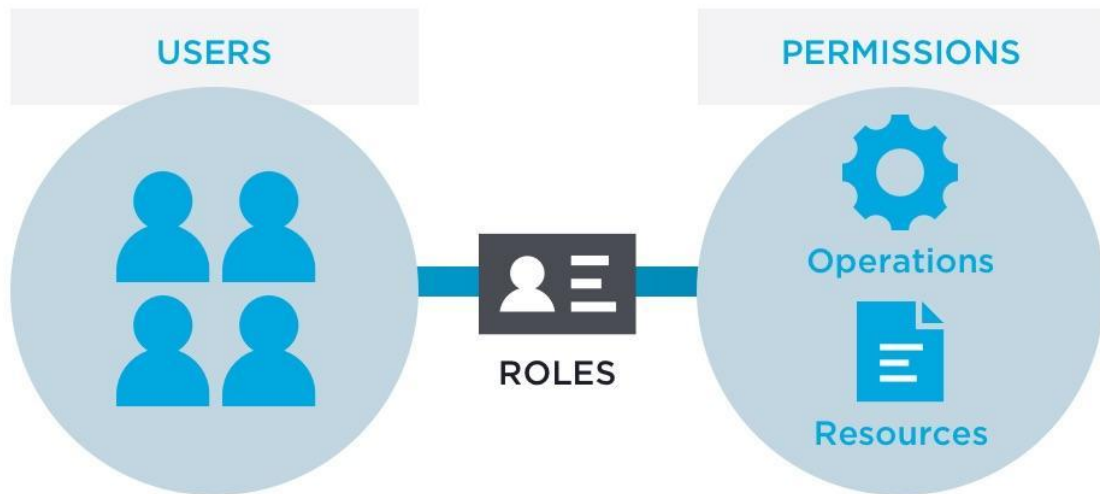
ROW-LEVEL SECURITY

ID	Region	Amount
1	West	3000
3		2500

Row Access Policies

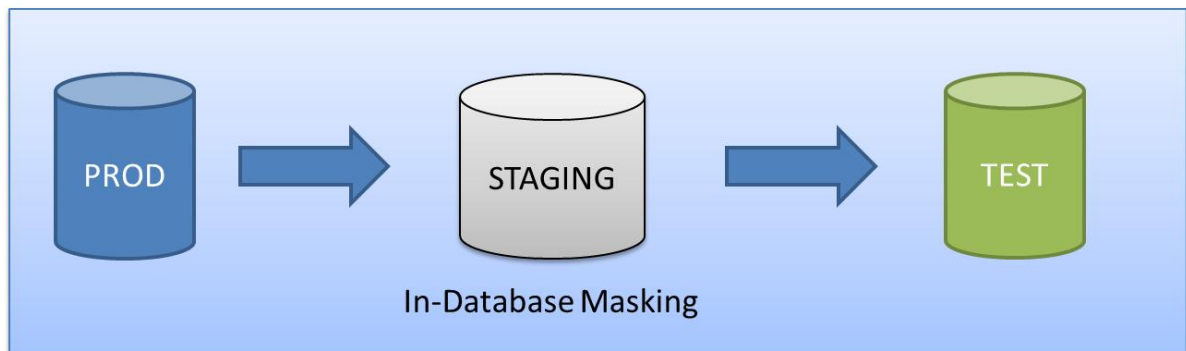
Limit access to specific rows



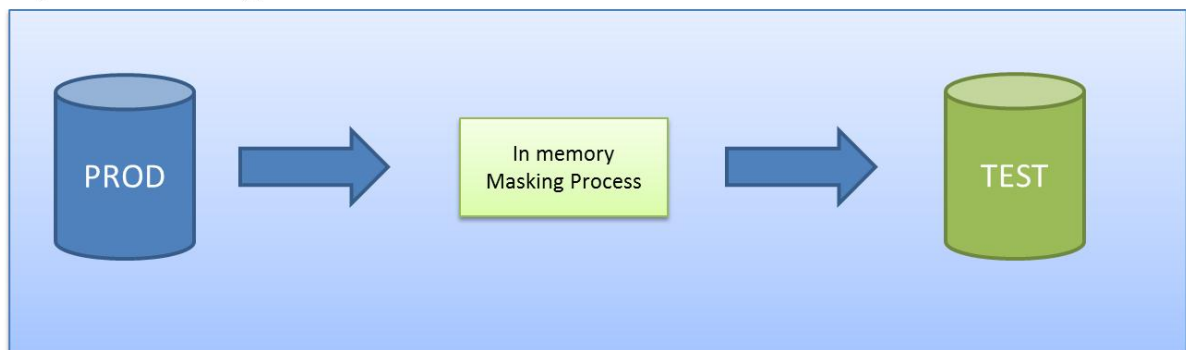


Before making masking dynamic and post making masking dynamic-

Static Masking



Dynamic Masking



3. Data Pipeline Automation & Change Tracking

Current Challenge:

- Manual ingestion of metadata- Metadata ingestion used to happen by manually doing a select * from query, then sampling the data using excel and manually applying the tags and doing classification. Data engineers identified sensitive fields based on business logic and rules. This process was time consuming and error prone.
- No proper tracking of historical changes- there was no tracking of history records. Once tagging done the information and data was not kept in any history table.
- Business dependency- only business teams knew the PII business rule book, what data is sensitive, context behind columns. Data engineers first discussed this with business teams every time for each new set of data.

Digitalisation Approach:

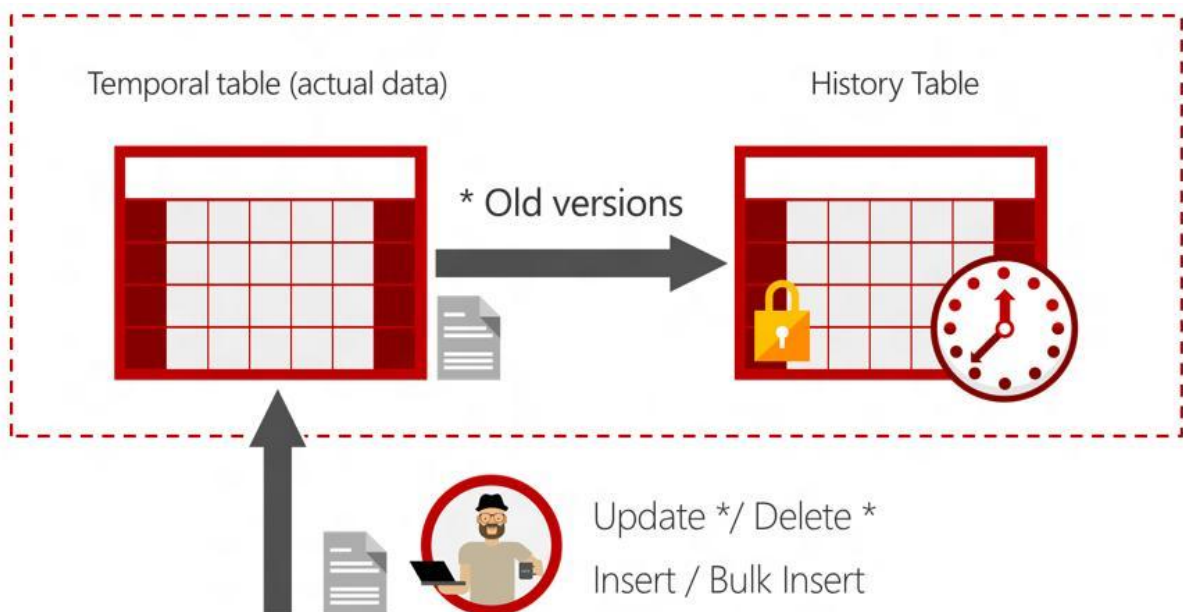
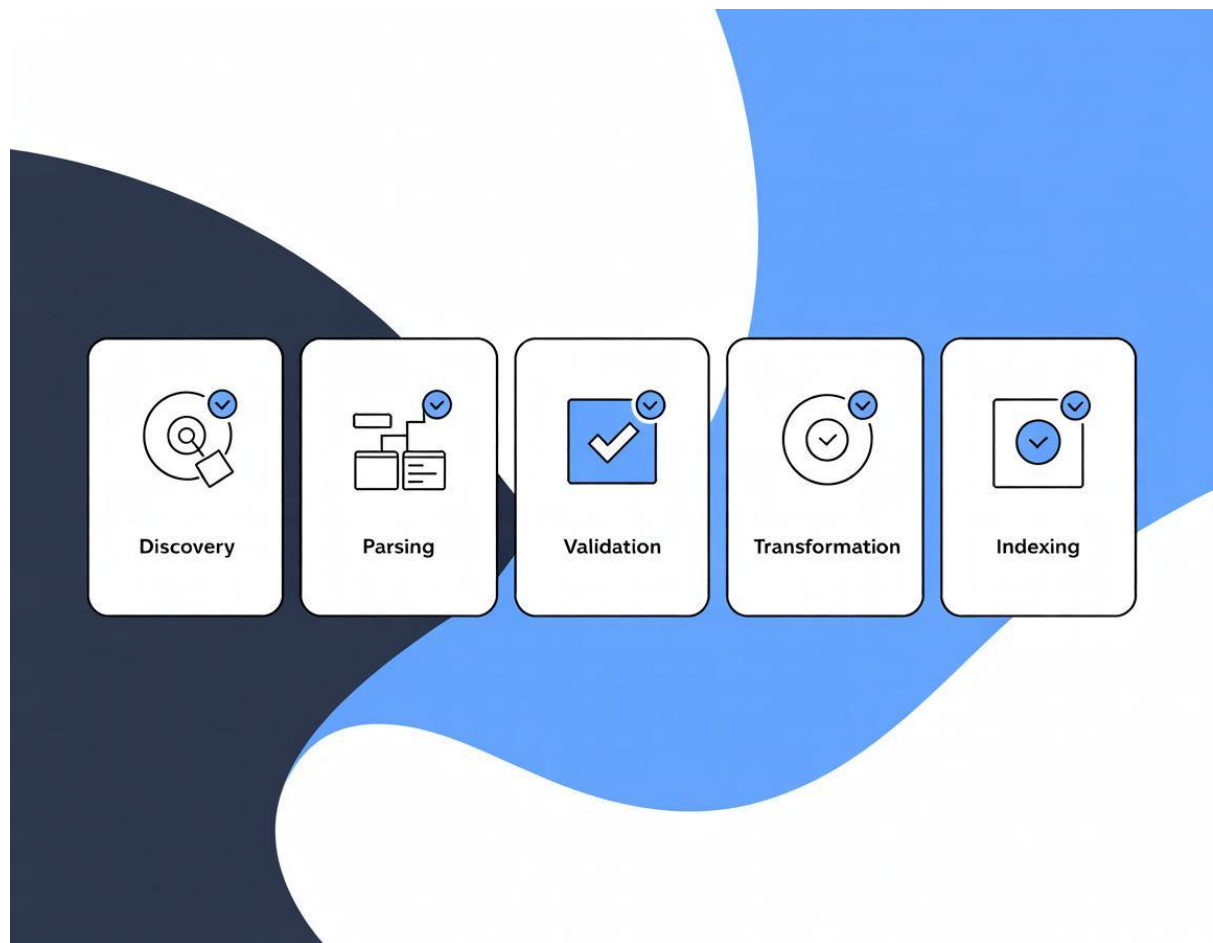
- API-based ingestion from Collibra – Snowflake- the Collibra Snowflake integration using custom API calls made for Snowflake to Collibra made processes not only smooth but free from manual errors. This became streamlined and adaptable very easily. We are also taking care of real time data updates and business partners are happy seeing data- safe, they are trusting us with our approach and this in turn is strengthening our customer relationships.
- Use staging and history tables:
 - LANDING which is current state
 - LANDING_HISTORY which is our historical tracking- we are now keeping a track of all our history loads so that inconsistency if any could be easily detectable and we can correct it. This history table is also connected to our data Presentation layer – Power BI for projecting the number of PII tagging done each day, week, month and so on.
- Automate via **Snowflake Tasks** - COLLIBRA_CLASSIFIEDMASKING_TASK. This task is responsible for triggering API and ingesting metadata from Collibra into Snowflake where latter all processing of tagging and application of dynamic masking policies happen automatically.

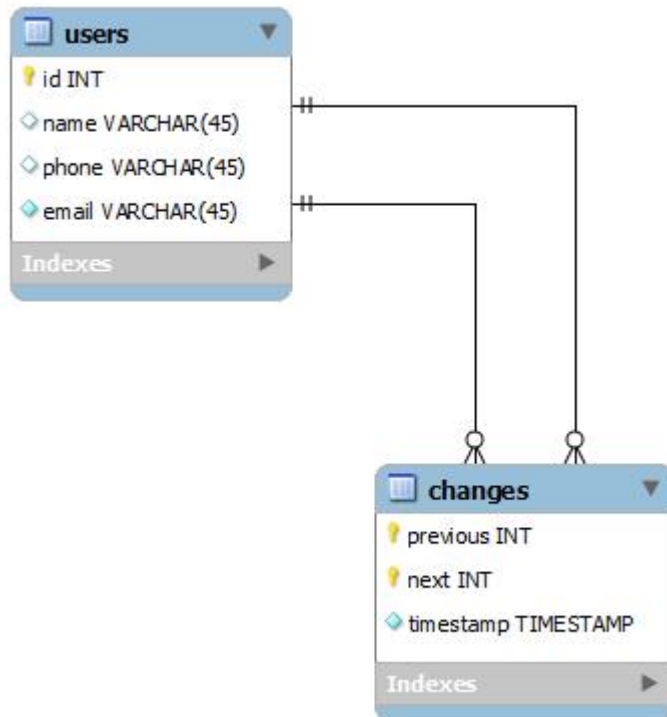
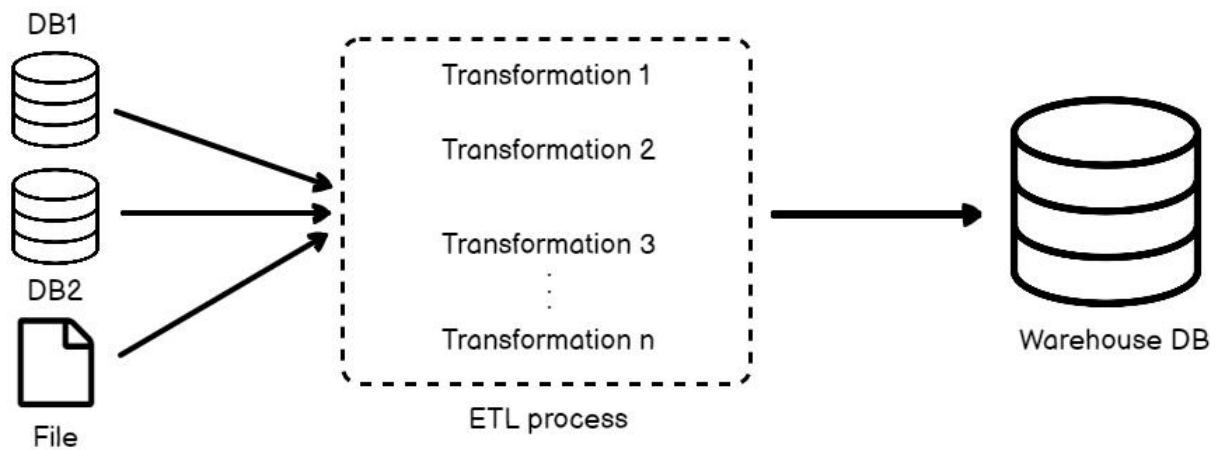
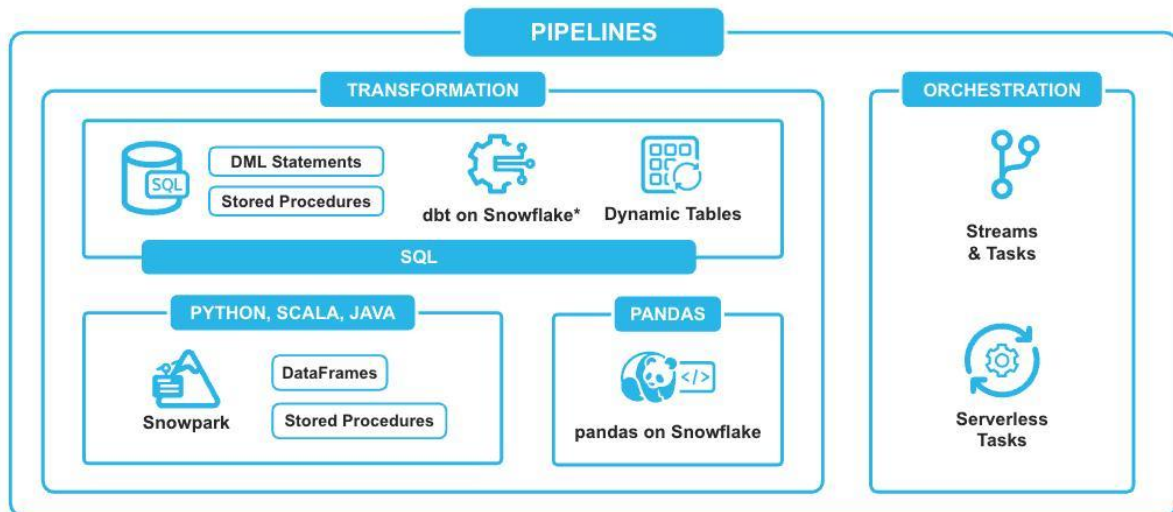
Outcome:

- End-to-end automation- the process is now smooth, streamlined and there is end to end automation at every step. Causing our governance structure robust and scalable. With time our data complexity grew and automation handled it seamlessly.
- Full audit trail of changes- we could easily go back and figure out the whys behind our data rules. This made us compliant and secure. When our data is secure, we are secure. There is more trust with us. We have full control over our policies.

Improved data transparency- with each automation we improved our data transparency. The metadata which resides in Collibra is completely in sync with our Snowflake staging tables and history tables.

Below figures illustrate the processes better of how we automated data pipeline and change tracking-





Snowflake Tasks Automation



Task 2: Challenges & Solutions

1. API Integration & Data Format Issues

Challenge:

- JSON inconsistencies from Collibra APIs- when we just started with our digitization approach we found initial inconsistencies in our JSON table. Reasons were-
 - Collibra metadata was not updated frequently. The why behind it is- Data Stewards were very less trained initially with the Collibra tool. When Data Stewards became compliant with the tool, periodic updating of metadata was maintained.
 - Incorrect username and password in API calls- the username and password was not updated in API procedures with the one used to access Collibra repository.
- Parsing errors in Snowflake- we faced initial parsing errors in Snowflake because of following reasons-
 - Invalid JSON format- When ingesting API data from Collibra we missed brackets { }, handle quotes improperly like JSON expects ' but we fetched and didn't correct ". For example Snowflake fails to parse this- { "name": "John", "age": 30, }
 - Inconsistent JSON structure- sometimes some records have some fields and other have different fields. For example nested structure vary here-
Record 1: { "email": "a@gmail.com" }
Record 2: { "email": { "work": "a@gmail.com" } }

- Null or Empty values- API returns null, empty strings, missing keys. Causes issues during transformation or casting

Solution:

- Standardize API response format- we then streamlined our every API response follows are standardized and consistent structure. This made Snowflake ingest API data from Collibra without any errors.
 - For example every record must follow this structure


```
{
  "schema_name": "SALES",
  "table_name": "CUSTOMER",
  "column_name": "EMAIL",
  "pii_flag": true,
  "data_type": "STRING",
  "source_system": "COLLIBRA"
}
```
 - Transform API response before loading- transform the raw API response using Python and apply SQL transformations to it after loading into Snowflake.
- Use robust JSON parsing logic in procedures API_TO_JSON- this is our very first API procedure which is responsible to make calls to Collibra and ingest JSON data to Snowflake. When we used proper JSON parsing logic here our API calls became error free and smooth.
- Handle edge cases like empty schema/table names- Handle missing and empty fields- when API is returning empty or null values setting defaults and using nulls safely is the next we incorporated.

2. Complex Historical Data Handling

Challenge:

- Maintaining accurate PII_ACTIVE status- initially we relied on one PII status as single point of status change. We found it difficult to maintain as history records were also to be changed and any change in old records the new PII status was causing confusion. So then we created PII_ACTIVE as another field taking care of our changed old records and maintaining history properly.
- Issues due to truncate-load process- so each time our staging table is getting truncated and getting loaded with new data. Initially we faced issues here as we needed history records. So we then introduced LANDING_HISTORY to take care of previous data and keep an easy set of records.

Solution:

- Introduce **persistent staging table LANDING_STAGE**- where all our records from JSON table were loaded. The JSON table has actual JSON records which actually appear in JSON format

and is uneasy to read and understand. So we introduced **LANDING_STAGE** to transform these JSON records in a tabular readable format.

- Enhance logic in MAINTAIN_HISTORY_LOADS- this is our stored procedure to load data in LANDING_HISTORY table for easy maintaining of History records.
 - Compare using COLLIBRACOLUMNNAME + TABLE_NAME- this is a **key design decision** in our solution which is using a **composite key of** COLLIBRACOLUMNNAME + TABLE_NAME to uniquely identify each record when comparing LANDING table data which is latest data with LANDING_HISTORY table data which is existing data records.
- Ensure no unintended updates to historical records- when my procedures are in sync my tables like staging table and history tables are always in sync. The procedures are triggered such that they are running on sequence basis – JSON to LANDING to LANDING_HISTORY and triggering only when JSON has relevant data.

3. Fine-Grained Access Control Complexity

Challenge:

- Managing access across AZCA and AWS environments- this was initially very complex until we segregated our governance models. Same governance was difficult to maintain because
 - One environment has role based access control and other has role+ actor based access so same logic was not fitting both.
 - Policy enforcement logic became inconsistent due to both environments being different.

Solution:

- Separate governance models:
 - Centralised tags in AZCA
 - Decentralised tags in AWS
- Maintain access tables differently for both the environments-
 - CLASSIFICATION_ACCESS_DETAILS_AWS
 - ENTITLEMENT_DATA
- Automate entitlement mapping- automatically linking users/roles to the data they are allowed to access. So Entitlements = Who can see what data is sensitive Mapping = Connecting- User / Role - Data classification based on PII tags
- These mappings were integrated with Snowflake masking policies, enabling real-time, rule-based access control without manual intervention, thereby improving scalability, consistency, and governance.

4. Performance & Scalability

Challenge:

- Large metadata volume impacting performance- when our data became huge over a period of time this impacted our performance. The procedures were taking long time to execute and process and transform the data which further delayed our performance.

Solution:

- Optimize Snowflake queries- we made use of parallel processing for faster data loads and optimization. Python functions like `threadpoolexecutor()` made it easy for processing data parallelly.
- Use partitioning and clustering- making use of Snowflake automatic micro- partitioning and clustering made data easy to process and handle. Snowflake's automatic micro-partitioning made parallel processing easier. Snowflake clustering made data location easier by creating groups of related data together, reducing data scanning time and faster filtering and joins.
- Schedule incremental loads instead of full refresh- we did real time updates and did incremental loading of data. Incremental loading of data is when older data is updated with new and fresh changes, if any in metadata and the new data is also taken care of.

5. Cross-Team Coordination

Challenge:

- Dependency between governance, engineering, and business teams- though the process was automated there was lot of dependency on all above teams. A change in one system has to be communicated to other system, otherwise the former system would continue to operate on older data. For example when username and password is updated in Collibra it has to be communicated to Snowflake team so that new username and password is updated in API procedures.

Solution:

- Use Token-Based Authentication - for all passwords related queries this is one stop solution. Use OAuth / API tokens which has benefits of no password sharing, tokens can expire & rotate automatically and are more secure.
- Define clear ownership- a clear ownership of processes and tasks makes governance easy and streamlined.
 - Collibra → Governance team
 - Snowflake → Data engineering
- Establish SLA-driven workflows- timelining and fixing the workflows, like this task should be completed in n minutes. For example,
 - **Collibra Classification Update** – SLA- *Within 24 hours of data onboarding* and responsibility- Governance team

- **API Ingestion to Snowflake-** SLA- *Within 1–2 hours after Collibra update*. Automated via- API_TO_JSON and Snowflake Tasks
- Data Load from LANDING table to HISTORY , SLA: Near real-time / scheduled e.g., every 30 minutes

Task 3: Framework for Digitalisation

Agile + Design Thinking

Step 1: Empathize & Define for Design Thinking

- Identify pain points to set the processes smooth for further refining.
 - Manual tagging- this was how the process were executing earlier. There was manual tagging, causing delayed workflow and data security was at question here.
 - Delayed masking- this further resulted in delayed masking. Again the masking logic was being applied manually.
 - Compliance risks- this bought lot of compliance risks. Governance was at a question, processes and proper documentation was being questioned.

Example: Inconsistent PII tagging across datasets. All these pain points not only were time consuming but also error prone. It made entire governance slow and unstructured. This led to design a solution which is structured, robust, scalable, error free and streamlined. If one thing goes here where we know exactly what to look at, what to fix within a day. Everything in our control and privacy well maintained and fixed.

Step 2: Ideation

- Design solution- we designed a solution which is automated, refined, structured and easy to follow. Creating not just responsible systems but also talent who know their tools and best practices.
 - Collibra-driven classification- when data classification was done in Collibra, PII was never missed. This resulted in a proper data ingestion and latter tags and dynamic masking policies were applied to the sensitive data. The processes got arranged and were executed based on scheduled tasks and real time updates.
 - Snowflake-based automation- all the logic to tag and mask the data was applied in Snowflake stored procedures. This made computation faster by making use of Snowflake's powerful computing platform, inbuilt micro-partitions, optimization and clustering to refine and tag the sensitive data.
 - Tag-based masking framework- again through Snowflake's powerful platform tagging and dynamic masking was easily possible and carried out making the entire workflow structured and automated.

Step 3: Build Agile Sprints

Break into iterations- we took initial 2 months for complete development and latter build iterations to refine our logic and make them robust. Since we follow agile methodology we created various sprints each of 2 weeks to help us build our governance workflow in a structured and automated way.

- **Sprint 1:** API ingestion to JSON_TABLE- sprint 1 focuses on data ingestion process to get streamlined. Here data engineers build proper store procedures and tasks to apply the logic so that our data ingestion is smooth and error free.
- **Sprint 2:** Landing & history framework- sprint 2 focused on creating the framework for staging table data load to history and maintain it further. So data engineers implemented **truncate-load mechanism** in Snowflake to ensure latest metadata from Collibra is always available. History table design was fixed. Created history table structure to maintain- PII_ACTIVE, START_DATE, END_DATE, UPDATED_DATE. This helped us to track **data changes over time and maintain audit trail**.
- **Sprint 3:** Tagging automation- in order to apply data classification tags in Snowflake data engineers created the tag Framework Setup in Snowflake, designed two models- **AZCA for Central tags stored in CONFIGURATION_AZCA_DB** and **AWS for Local tags stored** within each database. This established a strong foundation for automated data classification. Data Engineers then applied proper logic to determine which tag should be applied to which column. Removed need for manual tagging by automated Tag Application Logic by writing python stored procedures to identify columns needing tags and apply tags dynamically.
- **Sprint 4:** Masking policy implementation- further sprint 4 was all about building dynamic masking policies where the team implemented various case statements to support masking policies based on accounts and users pointing to ENTITLEMENT table where all accounts and users information was kept. These policies were created for all relevant datatype which the system encountered. This sprint were further to integrate masking policies with Tagging Framework where masking policies was applied **via tags**. Example- if a column email tagged as PII , automatically linked to masking policy VARCHAR.

Step 4: Test & Validate

- **Validate-** we had a separate QA team to validate all the tagging of sensitive data, once the framework was pushed to QA environment. QA analyst were also operating on sprints where they were taking up sets of databases to manually validate initially and latter they too developed automated testing using selenium for doing the validations.
 - **Correct tagging-** QA validate whether tags applied are correct based on sensitivity and classification in governance tool. This made our logic stronger to support our automation and also with each test case pass we build a more stronger governance framework.
 - **Proper masking behavior-** whether the masking is properly linked to its tag? Is there any difference in datatype of object and masking policy? All these questions were validated and again the test cases was passed for production deployment.
- **Test edge cases:**

- Empty schema/table- team validated the nulls and empty use cases where no schema or table meant no context.
- Multiple environments- since Rogers have 2 environments – AZCA and AWS, the QA team tested all the scenarios for both the environments. The team validated environment-specific entitlement handling, testing across environments.

Step 5: Deploy & Automate

- Deploy Snowflake Tasks- we followed deployment of all Snowflake tasks for a smoother process flow. Once the stored procedures were ready they were called via tasks and once the tasks were triggered the entire framework was set to flow and process customers sensitive PII data. This made our governance stronger and smooth.
- Automate end-to-end pipeline- team build data pipelines to run the code in all the environments- Dev, Test, and prod. This makes all the environments to be in sync and follow same processes across all. CI/CD pipelines are triggered on the deployment day. We have separate set of operations team and release engineers for this activity.
- Enable monitoring- once code is deployed in the respective environment we have to monitor the tasks and check for the real time updates the code is pushing.

Step 6: Continuous Improvement

- Track:
 - Data accuracy- the level of data accuracy determines the correctness of code, build logic which further paves a way for consistency in metadata classification and tags applied in Snowflake. It is very important for the two to be always in sync.
 - Policy effectiveness- the real time policy application is another major factor to be traced for. The dynamic masking policy should be in sync with the tag applied to the database object and should have followed all the business logic and masking rules.
- Enhance logic e.g. new PIFAMILYGROUP rules- we have build our logic such that for any new changes in the PII either addition of a key and value pair or deletion or any update for that matter, only the base table needs to be changed. Our core logic and stored procedures remain the same. This design helps us in improved scalability, maintainability, and adaptability of the data governance framework.

Conclusion

The digitalisation of core data governance functions at Rogers Communications using Collibra and Snowflake enables-

- **Automated data classification and tagging-** a major shift from manual approach, which makes the entire workflow robust, scalable, autonomous, error free, streamlined and structured which forms Rogers integral data governance framework stand out and trust worthy among clients, users, partners.

- **Dynamic, policy-driven data masking**- the run time application of policy is our another core differentiator which makes the governance workflow smooth and compliant.
- **End-to-end pipeline automation with historical traceability**- another factor which is capable of keeping a track of all history records for better audits and smooth system integration following seamless CI/CD pipeline execution.

This approach ensures-

- Regulatory compliance- the GDPR and HIPAA regulations are well followed which makes Rogers compliant.
- Scalable architecture- if in future any new modification or addition of PII has to be made it will be just a table change and not any part of core code or core logic. We experienced recently huge volumes of data coming in for masking. When our core logic was strong we were able to not just handle the voluminous data but also process and compute it through our powerful processing and computing framework and masking algorithms.
- Strong data governance framework- all of this facilitated a strong governance foundation which resulted in more projects coming in from Rogers to Capgemini and happy clients.

Ultimately transforming Rogers into a **data-driven, secure, and compliant digital enterprise** which is governance friendly, structured and fully automated.